

EC200U&EG91xU Series

QuecOpen(SDK)

Camera Development Guide

LTE Standard Module Series

Version: 1.0

Date: 2023-11-08

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2022-01-13	Chaos HUANG	Creation of the document
1.0	2023-11-08	Richard LIANG	First official release

Contents

About the Document.....	3
Contents	4
Table Index.....	5
Figure Index	6
1 Introduction	7
2 Camera API.....	8
2.1. Header File	8
2.2. API Overview	8
2.3. API Description	9
2.3.1. ql_CamInit	9
2.3.1.1. ql_errcode_camera_e.....	9
2.3.2. ql_CamPowerOn.....	11
2.3.3. ql_CamPowerOff.....	12
2.3.4. ql_CamClose.....	12
2.3.5. ql_CamGetSensorInfo.....	12
2.3.5.1. ql_cam_drv_s	13
2.3.5.2. ql_cam_status_s.....	13
2.3.6. ql_CamPreview	14
2.3.7. ql_CamStopPreview.....	14
2.3.8. ql_CamCaptureImage	15
2.3.9. ql_camPrint	15
2.3.10. ql_CamSetBufopt.....	15
2.3.10.1. ql_buf_config_e	16
2.3.11. ql_CamInit0310SPI	16
2.3.12. ql_CamPreviewDQBUF	17
2.3.13. ql_CamPreviewQBUF	17
2.3.14. ql_CamI2CRead	18
2.3.15. ql_CamI2CWrite.....	18
3 Development Example and Debugging	19
3.1. Development Example	19
3.2. Feature Debugging.....	20
4 Appendix References	22

Table Index

Table 1: API Overview 8

Table 2: Related Documents 22

Table 3: Terms and Abbreviations 22

Figure Index

Figure 1: ql_camera_app_init()	19
Figure 2: Add Comments to ql_camera_app_init()	20
Figure 3: Camera Log 1	21
Figure 4: Camera Log 2	21

1 Introduction

Quectel EC200U series and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces camera API, development example and feature debugging on Quectel EC200U series and EG91xU family (EG912U-GL and EG915U series) modules in QuecOpen® solution.

2 Camera API

2.1. Header File

ql_api_camera.h, the header file of camera API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all header files mentioned in this document are in this directory.

2.2. API Overview

Table 1: API Overview

Function	Description
<i>ql_CamInit()</i>	Initializes camera.
<i>ql_CamPowerOn()</i>	Turns camera on.
<i>ql_CamPowerOff()</i>	Turns camera off, without releasing camera resources.
<i>ql_CamClose()</i>	Turns camera off, and releases its camera resources.
<i>ql_CamGetSensorInfo()</i>	Gets camera configuration information.
<i>ql_CamPreview()</i>	Enables preview.
<i>ql_CamStopPreview()</i>	Stops preview.
<i>ql_CamCaptureImage()</i>	Uses camera to take a single photo.
<i>ql_camPrint()</i>	Outputs images to LCD.
<i>ql_CamSetBufopt()</i>	Sets the number of camera buffers.
<i>ql_CamInit0310SPI()</i>	Initializes the 0310 camera that communicates via SPI.
<i>ql_CamPreviewDQBUF()</i>	Gets the image data during camera preview.

<code>ql_CamPreviewQBUF()</code>	Ensures that the <code>ql_CamPreviewDQBUF()</code> gets image data during camera preview normally.
<code>ql_CamI2CRead()</code>	Reads camera register data via I2C.
<code>ql_CamI2CWrite()</code>	Writes data to the camera registers via I2C.

2.3. API Description

2.3.1. ql_CamInit

This function initializes camera.

- **Prototype**

```
ql_errcode_camera_e ql_CamInit(uint16_t width, uint16_t height)
```

- **Parameter**

width:

[In] Image width.

height:

[In] Image height.

- **Return Value**

See **Chapter 2.3.1.1** for details.

NOTE

Currently, the module supports a 300,000-pixel camera with the maximum image size can be 640 × 480.

2.3.1.1. ql_errcode_camera_e

The enumeration of result codes:

```
typedef enum
{
    QL_CAMERA_SUCCESS = QL_SUCCESS,
```

```

QL_CAMERA_INIT_ERR                = 1|QL_CAMERA_ERRCODE_BASE,
QL_CAMERA_ALREADY_INIT_ERR,
QL_CAMERA_NO_INIT_ERR,
QL_CAMERA_POWER_ON_ERR            = 5|QL_CAMERA_ERRCODE_BASE,
QL_CAMERA_ALREADY_POWER_ON_ERR,
QL_CAMERA_CLOSE_ERR,
QL_CAMERA_PREVIEW_ERR             = 10|QL_CAMERA_ERRCODE_BASE,
QL_CAMERA_PREVIEWING_ERR,
QL_CAMERA_NO_PREVIEW_ERR,
QL_CAMERA_STOP_PREVIEW_ERR,
QL_CAMERA_CAPTURE_ERR,
QL_CAMERA_GET_INFO_ERR,
QL_CAMERA_LCDINFO_GET_ERR,
QL_CAMERA_PRINT_ERR,
QL_CAMERA_DQBUF_ERR,
QL_CAMERA_QBUF_ERR,
QL_CAMERA_SET_BUF_ERR,
QL_CAMERA_CREATE_MUTEX_ERR,
QL_CAMERA_MUTEX_ALREADY_EXIST_ERR,
QL_CAMERA_MUTEX_TIMEOUT_ERR,
QL_CAMERA_DELETE_MUTEX_ERR,
QL_CAMERA_I2C_READ_ERR,
QL_CAMERA_I2C_WRITE_ERR,
}ql_errcode_camera_e

```

● Member

Member	Description
<code>QL_CAMERA_SUCCESS</code>	Successful execution.
<code>QL_CAMERA_INIT_ERR</code>	Failed to initialize camera.
<code>QL_CAMERA_ALREADY_INIT_ERR</code>	Initialized.
<code>QL_CAMERA_NO_INIT_ERR</code>	Not initialized.
<code>QL_CAMERA_POWER_ON_ERR</code>	Failed to turn camera on.
<code>QL_CAMERA_ALREADY_POWER_ON_ERR</code>	Powered on.
<code>QL_CAMERA_CLOSE_ERR</code>	Failed to turn camera off.
<code>QL_CAMERA_PREVIEW_ERR</code>	Preview failure.
<code>QL_CAMERA_PREVIEWING_ERR</code>	Previewing.

QL_CAMERA_NO_PREVIEW_ERR	Not in preview.
QL_CAMERA_STOP_PREVIEW_ERR	Failed to stop preview.
QL_CAMERA_CAPTURE_ERR	Failed to take photos.
QL_CAMERA_GET_INFO_ERR	Failed to get camera information.
QL_CAMERA_LCDINFO_GET_ERR	Failed to get LCD information.
QL_CAMERA_PRINT_ERR	Failed to output an image to LCD.
QL_CAMERA_DQBUF_ERR	Failed to get data frame.
QL_CAMERA_QBUF_ERR	Failed to transmit data to buffer.
QL_CAMERA_SET_BUF_ERR	Failed to configure buffer.
QL_CAMERA_CREATE_MUTEX_ERR	Failed to create Mutex.
QL_CAMERA_MUTEX_ALREADY_EXIST_ERR	The mutex already exists.
QL_CAMERA_MUTEX_TIMEOUT_ERR	The mutex lock expired.
QL_CAMERA_DELETE_MUTEX_ERR	Failed to delete Mutex.
QL_CAMERA_I2C_READ_ERR	I2C read error.
QL_CAMERA_I2C_WRITE_ERR	I2C write error.

2.3.2. ql_CamPowerOn

This function turns camera on.

- **Prototype**

```
ql_errcode_camera_e ql_CamPowerOn(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.3. ql_CamPowerOff

This function turns camera off, without releasing its resources. This function is only suitable for scenes where the camera is frequently turned on or off. To release camera resource, call *ql_CamClose()*, and there is no need to call *ql_CamPowerOff()*.

- **Prototype**

```
ql_errcode_camera_e ql_CamPowerOff(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.4. ql_CamClose

This function turns camera off, and release its resources.

- **Prototype**

```
ql_errcode_camera_e ql_CamClose(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.5. ql_CamGetSensorInfo

This function gets camera configuration information.

- **Prototype**

```
ql_errcode_camera_e ql_CamGetSensorInfo(ql_cam_drv_s *pCamDevice)
```

- **Parameter**

pCamDevice:

[Out] Camera configuration information. See **Chapter 2.3.5.1** for details.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.5.1. ql_cam_drv_s

The structure of camera configuration information is defined as below:

```
typedef struct
{
    char *pNamestr;
    uint32_t img_width;
    uint32_t img_height;
    uint32_t nPixels;
    ql_cam_status_s Status;
} ql_cam_drv_s
```

- **Parameters**

Type	Parameter	Description
char *	<i>pNamestr</i>	Camera name.
uint32_t	<i>img_width</i>	Image width.
uint32_t	<i>img_height</i>	Image height.
uint32_t	<i>nPixels</i>	Image pixels. Maximum pixel count: 614,400 bytes.
<i>ql_cam_status_s</i>	<i>Status</i>	Camera status. See Chapter 2.3.5.2 for details.

2.3.5.2. ql_cam_status_s

The structure of camera status is defined as below:

```
typedef struct
{
    bool Cam_Init_status;
    bool Cam_Power_status;
```

```
bool Cam_Preview_status;
} ql_cam_status_s
```

● Parameters

Type	Parameter	Description
bool	<i>Cam_Init_status</i>	Initialized.
bool	<i>Cam_Power_status</i>	Powered.
bool	<i>Cam_Preview_status</i>	Preview.

2.3.6. ql_CamPreview

This function enables preview.

● Prototype

```
ql_errcode_camera_e ql_CamPreview(void)
```

● Parameter

None

● Return Value

See **Chapter 2.3.1.1** for details.

2.3.7. ql_CamStopPreview

This function stops preview.

● Prototype

```
ql_errcode_camera_e ql_CamStopPreview(void)
```

● Parameter

None

● Return Value

See **Chapter 2.3.1.1** for details.

2.3.8. ql_CamCaptureImage

This function uses camera to take a single photo.

- **Prototype**

```
ql_errcode_camera_e ql_CamCaptureImage(uint16_t **pFrameBuf)
```

- **Parameter**

pFrameBuf:

[Out] Pointer to the saved photo.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.9. ql_camPrint

This function outputs images to LCD.

- **Prototype**

```
ql_errcode_camera_e ql_camPrint(uint16_t *imageBuf)
```

- **Parameter**

imageBuf:

[In] Displayed image. To display an image preview, enter *NULL* for this parameter.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.10. ql_CamSetBufopt

This function sets the number of camera buffers.

- **Prototype**

```
ql_errcode_camera_e ql_CamSetBufopt(ql_buf_config_e buf_cfg)
```


- **Parameter**

buf_cfg:

[In] Camera buffer count. See **Chapter 2.3.10.1** for details.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.10.1. ql_buf_config_e

The enumeration of camera buffers:

```
typedef enum
{
    single_buf,
    double_buf,
}ql_buf_config_e
```

- **Member**

Member	Description
<i>single_buf</i>	Single buffer.
<i>double_buf</i>	Double buffers.

2.3.11. ql_CamInit0310SPI

This function initializes 0310 camera that communicates via SPI.

- **Prototype**

```
ql_errcode_camera_e ql_CamInit0310SPI(uint16_t width, uint16_t height)
```

- **Parameter**

width:

[In] Camera width.

height:

[In] Camera height.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.12. ql_CamPreviewDQBUF

This function gets image data during camera preview. This function must be used in conjunction with *ql_CamPreviewQBUF()*, following the order of calling *ql_CamPreviewDQBUF()* first, then calling *ql_CamPreviewQBUF()*.

- **Prototype**

```
ql_errcode_camera_e ql_CamPreviewDQBUF(uint16_t **pFrameBuf)
```

- **Parameter**

pFrameBuf:

[Out] Image data.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.13. ql_CamPreviewQBUF

This function ensures that the *ql_CamPreviewDQBUF()* gets image data during camera preview normally. This function must be used in conjunction with *ql_CamPreviewDQBUF()*, following the order of calling *ql_CamPreviewDQBUF()* first, then calling *ql_CamPreviewQBUF()*.

- **Prototype**

```
ql_errcode_camera_e ql_CamPreviewQBUF(uint16_t *pFrameBuf)
```

- **Parameter**

pFrameBuf:

[In] Image data.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.14. ql_CamI2CRead

This function reads camera register data via I2C.

- **Prototype**

```
ql_errcode_camera_e ql_CamI2CRead(uint8_t addr, uint8_t *data, uint32_t len)
```

- **Parameter**

addr:

[In] Register address.

data:

[Out] Register data.

len:

[In] Data length.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.15. ql_CamI2CWrite

This function writes data to the camera registers via I2C.

- **Prototype**

```
ql_errcode_camera_e ql_CamI2CWrite(uint8_t addr, uint8_t data)
```

- **Parameter**

addr:

[In] Register address.

data:

[Out] Register data.

- **Return Value**

See **Chapter 2.3.1.1** for details.

3 Development Example and Debugging

3.1. Development Example

camera_demo.c, the camera API demo file, is provided in *components\ql-application\camera* in CSDK directory of Quectel EC200U series and EG91xU family (EG912U-GL and EG915U series) modules. The file outlines how to initialize, turn on and turn off the camera, as well as perform various camera operations.

The example begins with initializing the camera, turning on the camera and activating its preview feature, and taking 10 photos after a 20-second preview. If the camera is connected to an LCD, the images will be successively displayed on the LCD. After the photos are taken, the preview stops, and the camera is turned off.

The entry function is *ql_camera_app_init()*, as shown below:

```
void ql_camera_app_init(void)
{
    QLCAMERAStatus err = QL_OSI_SUCCESS;
    ql_task_t camera_task = NULL;

    err = ql_rtos_task_create(&camera_task, QL_CAMERA_TASK_STACK_SIZE, QL_CAMERA_TASK_Prio, "camera DEMO", ql_camera_demo_thread, NULL, QL_CAMERA_TASK_EVENT_CNT);
    if (err != QL_OSI_SUCCESS)
    {
        QL_CAMERADEMO_LOG("CAMERA demo task created failed");
    }
}
```

Figure 1: *ql_camera_app_init()*

As shown below, the above example is disabled by default in *ql_init_demo_thread()*. Uncomment *ql_camera_app_init()* if you need to test it. Note that the camera does not work properly because some of the pins required for camera functionality may overlap with the pins used in *ql_ledcfg_app_init()*. Therefore you need to comment *ql_ledcfg_app_init()* when using this example. In addition, when running this example, be aware that both the I2C and decoder API example files require camera access. As a result, you cannot call simultaneously *ql_i2c_demo_init()* and *ql_decoder_app_init()* when running the example. Otherwise the three example files will conflict in their camera usage and may even cause a module crash.

```

20:
21: static void ql_init_demo_thread(void *param)
22: {
23:     QL_INIT_LOG("init demo thread enter, param 0x%x", param);
24:     #if 0
25:         ql_gpio_app_init();
26:         ql_gpioint_app_init();
27:     #endif
28:
29:     #ifdef QL_APP_FEATURE_CAMERA
30:         //ql_camera_app_init();
31:     #endif
32:
33:     #ifdef QL_APP_FEATURE_LEDCFG
34:         ql_ledcfg_app_init();
35:     #endif
36:
37:     #ifdef QL_APP_FEATURE_AUDIO
38:         ql_audio_app_init();
39:     #endif
40:     #ifdef QL_APP_FEATURE_LCD
41:         //ql_lcd_app_init();
42:     #endif
43:     ql_nw_app_init();
44:     //ql_datacall_app_init();

```

Figure 2: Add Comments to ql_camera_app_init()

3.2. Feature Debugging

Camera of EC200U series and EG91xU family (EG912U-GL and EG915U series) modules can be debugged on Quectel LTE OPEN EVB.

The debug procedure is shown below:

1. Download the compiled firmware to the module.
2. Connect the USB port of the LTE OPEN EVB to the PC with a USB cable.
3. Capture the log with the log capture tool “cooltools” via USB AP Log Port. For details about log capture, see **document [2]**.
4. The USB AP Log Port mainly displays the system debugging information. After you capture the log, you will be able to check the development related information.

ql_camera_app_init() starts automatically after the module is booted. The debugging information outputted from AP Log ports is shown in **Figure 3** and **Figure 4**.

```
[ql_CAMERADEMO][ql_camera_demo_thread, 99] Camera demo entry
cam: quec_CamInit
cam: _drvCamLoad
cam: _drvCamGetCfgCount count=2
power_id = CAMD, step = 12500, min_uv = 1400000, max_uv = 2187500
power_id = CAMA, step = 12500, min_uv = 1612500, max_uv = 3200000
cam get id 0x23,0x2a
CamGc032aGetSensorInfo 80c0ba14
[ql_api_camera][ql_CamGetSensorInfo, 175] Get cam sensor Info
cam: drvCamGetSensorInfo
[ql_api_camera][ql_CamInit, 220] Cam init success
cam: drvCamPowerOn
camGc032aOpen
power_id = CAMD, step = 12500, min_uv = 1400000, max_uv = 2187500
power_id = CAMA, step = 12500, min_uv = 1612500, max_uv = 3200000
[ql_api_camera][ql_CamPowerOn, 81] Cam power on success
[ql_CAMERADEMO][ql_camera_demo_thread, 113] testID[0] = 0x0, testID[1] = 0x0
[ql_CAMERADEMO][ql_camera_demo_thread, 117] testID[0] = 0x23, testID[1] = 0x2a
cam: drvCamStartPreview
camGc032aPrevStart gc032aInfo.preview_page=0
[ql_api_camera][ql_CamPreview, 124] Cam preview success
[ql_api_camera][ql_camPrint, 238] Cam lcd width = 240 height = 320
```

Figure 3: Camera Log 1

```
[ql_api_camera][ql_CamCaptureImage, 165] Cam capture success
cam: drvCamCaptureImage
[ql_api_camera][ql_CamCaptureImage, 165] Cam capture success
cam: drvCamStopPreview
[ql_api_camera][ql_CamStopPreview, 148] Cam stop preview success
cam:drvCamClose,osisemaphoreDelete
```

Figure 4: Camera Log 2

4 Appendix References

Table 2: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 3: Terms and Abbreviations

Abbreviation	Description
AP	Application Processor
API	Application Programming Interface
APP	Application
COM	Communication
EVB	Evaluation Board
GPIO	General-Purpose Input/Output
IoT	Internet of Things
LCD	Liquid Crystal Display
RTOS	Real-Time Operating System
SDK	Software Development Kit
SPI	Serial Peripheral Interface
USB	Universal Serial Bus